Projet de Programmation
Master Informatique
2025–2026

université de BORDEAUX

Quoridor – Java

Rapport final

Abdeldjabar Ismail Abdraman, Cengiz Vasseur, Joris Douillet, Khadidja Ahmat Hassan, Matthieu Pourageaud

2 avril 2026

1 Architecture du projet

1.1 Justification du choix d'architecture

Notre projet Quoridor nécessite l'implémentation d'un moteur de jeu respectant des règles strictes, tout en supportant plusieurs modes d'interaction (interface en ligne de commande, interface graphique, intelligence artificielle et jeu en réseau). Ces contraintes impliquent une forte complexité métier, une exigence de fiabilité des règles ainsi qu'une capacité d'évolution du logiciel. Afin de répondre à ces objectifs, nous avons retenu une architecture modulaire orientée domaine, inspirée de la Clean Architecture (Ports & Adapters) [2].

Notre système est organisé autour d'un cœur métier central (`core`) qui regroupe l'ensemble de la logique du jeu : règles, état de la partie, moteur d'exécution et entités principales. Ce module est totalement indépendant des autres, garantissant que la logique métier reste cohérente, testable et inchangée, quelles que soient les modalités d'interaction avec le jeu.

Les modules complémentaires (`cli`, `gui`, `network`, `persistence`, `config`, `i18n`) fonctionnent comme des adaptateurs qui interagissent avec le cœur via des interfaces bien définies. Cette organisation permet de modifier, ajouter ou supprimer des interfaces sans impacter la logique du jeu. Par exemple, l'IA (modules `ai/minimax` ou `ai/mcts`) peut simuler des parties en utilisant directement `GameState` et `Board`, sans dépendre de l'interface graphique ou de la CLI.

Le patron MVC (Model–View–Controller) est utilisé uniquement au niveau des interfaces. Le module `core` correspond au modèle, tandis que les modules `cli` et `gui` implémentent les vues et les contrôleurs. Cette utilisation locale de MVC permet de séparer clairement la présentation de la logique métier sans imposer ce modèle à l'ensemble de l'architecture.

Ce choix architectural apporte plusieurs bénéfices : une maintenabilité accrue, une testabilité élevée du moteur de jeu, une extensibilité facilitée (ajout de nouvelles IA ou interfaces) et une robustesse globale, puisque toutes les actions passent nécessairement par la validation des règles du cœur métier. Il constitue ainsi un compromis efficace entre rigueur logicielle, évolutivité et lisibilité du code, et s'avère adapté à un projet de jeu à règles complexes comme Quoridor.

1.2 Vue d'organisation des couches

La règle de dépendance est respectée : toutes les flèches pleines pointent vers l'intérieur (`core`), et jamais l'inverse.

Couche	Responsabilité
Couche 1	Domaine (core) : le noyau immuable, sans dépendances externes. Toutes les entités métier y résident.
Couche 2	Application (util, config, graph) : services transversaux proches du domaine. Le module graph fournit la liste d'adjacence utilisée par le RuleChecker pour valider les chemins.
Couche 3	Adaptateurs (cli, gui, network, ai, persistence, i18n) : les différents adaptateurs qui interagissent avec le monde extérieur tout en ne dépendant que du core.
Couche 4	Infrastructure / Entrée (Main) : orchestre le démarrage via le patron Factory, instancie le bon launcher (CLI, GUI ou Network) et reste complètement découplée du domaine.

TABLE 1 – Organisation des couches

1.3 Vue d'ensemble des packages

Le projet est organisé en onze packages distincts, chacun ayant une responsabilité bien définie :

Package	Responsabilité
core	Entités métier : Board, GameState, Player, Move, Position, GameEngine, RuleChecker
Main	Point d'entrée, résolution du mode de lancement, patron Factory
cli	Interface en ligne de commande avec le pattern Command pour chaque action utilisateur
gui	Interface graphique comportant les menus 'File' et 'Game' (JavaFX), proposant des raccourcis clavier. L'utilisateur doit pouvoir définir ses propres raccourcis pour chaque action listée
network	Implémente les commandes réseau de base et étendues pour gérer plusieurs clients simultanément, en utilisant un thread par client ou de la programmation asynchrone. Serveur TCP avec découverte UDP et client réseau
ai	Joueurs IA : Random, Minimax (alpha-beta, iterative deepening), MCTS, réseau de neurones
persistence	Sérialisation/désérialisation de l'état de jeu vers un fichier texte
util	Logger singleton partagé dans toute l'application
i18n	Gestion de la langue du logiciel. Si les variables d'environnement 'LANG' ou 'LC_ALL' sont définies, le programme doit les utiliser pour déterminer la langue à utiliser
graph	Module Graph et GraphBuilder implémenté sous la forme d'une liste d'adjacence (adjacency list) pour représenter l'état du plateau de jeu
config	Fichier de configuration nommé '.quoridorrc', placé à la racine du répertoire de l'utilisateur (HOME). Le fichier est lu au démarrage du programme s'il est présent, et les options qu'il contient sont utilisées par défaut. Sinon, un fichier de configuration minimal est créé automatiquement

TABLE 2 – Vue d'ensemble des packages

1.4 Vue en couches concentriques

schema en couches concentriques de notre architecture du projet (Fig. 1).

1.5 Couche Domaine — Package `core`

Le package `core` constitue le cœur du système. Il contient l'ensemble des entités métier ainsi que les règles du jeu. Aucune dépendance vers des bibliothèques externes ou d'autres couches n'est autorisée ici, garantissant une testabilité et une portabilité maximales.

schema du couche core de notre architecture(Fig. 2).

1.6 Couche Principale — Package `Main`

Le package `Main` est le point d'entrée de l'application. Il illustre l'utilisation conjointe des patrons **Factory** et **Strategy** pour sélectionner dynamiquement le mode de lancement en fonction des arguments passés en ligne de

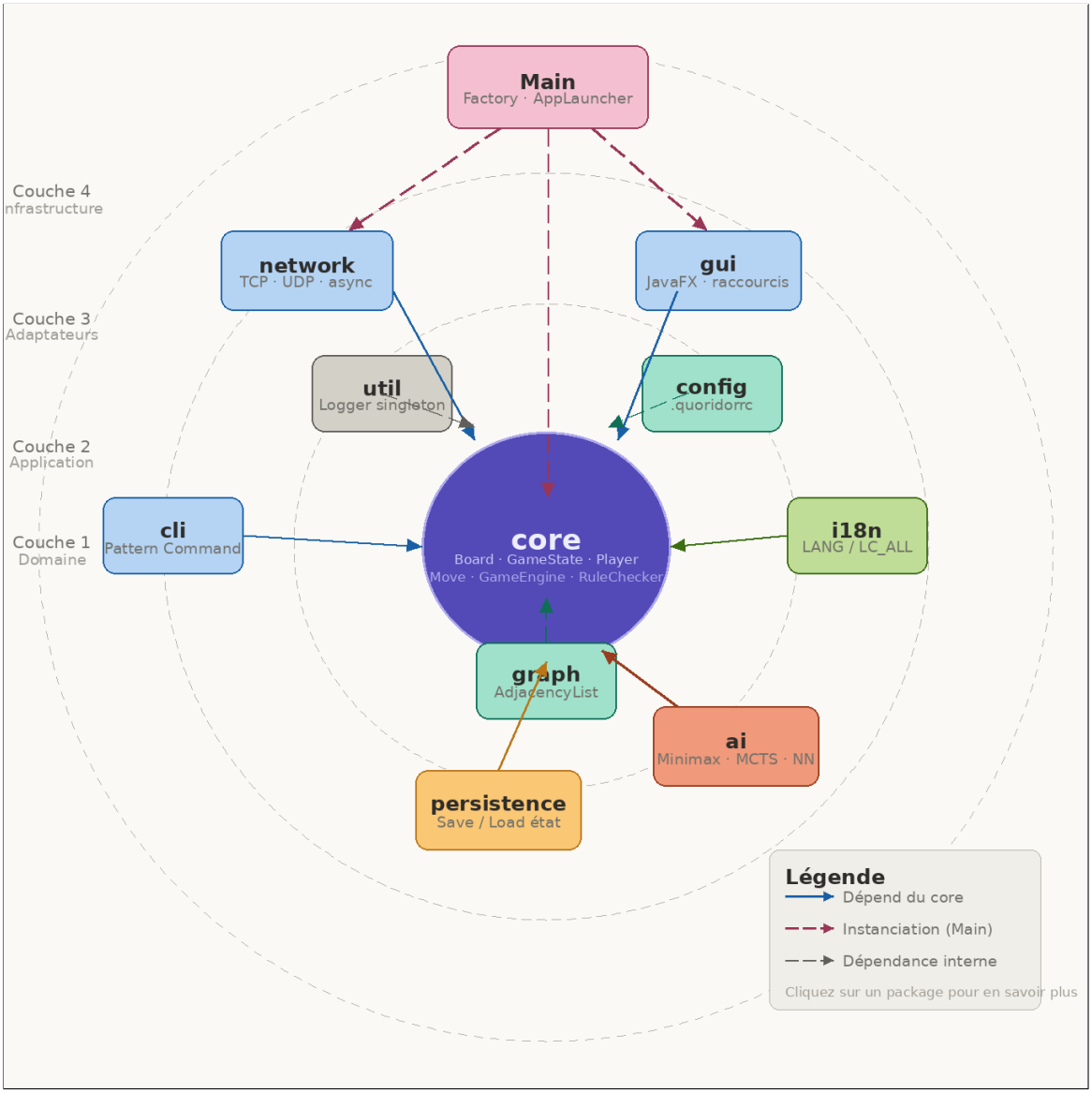


FIGURE 1 – Une architecture en couches

commande.

schema du couche main de notre architecture (Fig. 3).

1.7 Interface CLI — Pattern Command

Le package `cli` implémente l'interface en ligne de commande en utilisant le **patron Command**. Chaque action utilisateur est encapsulée dans une classe dédiée implémentant l'interface `Command`. Cela permet une meilleure extensibilité, une testabilité unitaire de chaque commande et facilite l'ajout d'un mécanisme d'annulation.

schema du couche cli de notre architecture (Fig. 4).

1.8 Module Réseau — Package `network`

Le module réseau permet des parties entre clients distants via TCP. La découverte des serveurs disponibles sur le réseau local s'effectue via des diffusions UDP (UDP broadcast), simplifiant l'expérience multijoueur sans configuration manuelle.

1.9 Points forts architecturaux

- **Indépendance du domaine** : le package `core` est du Java pur, sans dépendance externe
- **Extensibilité de l'IA** : tout nouvel algorithme étend simplement `AiPlayer`
- **Pattern Command en CLI** : chaque commande est testable et extensible indépendamment
- **Persistance découplée** : `GameSaver` / `GameLoader` ne connaissent que les interfaces du `core`

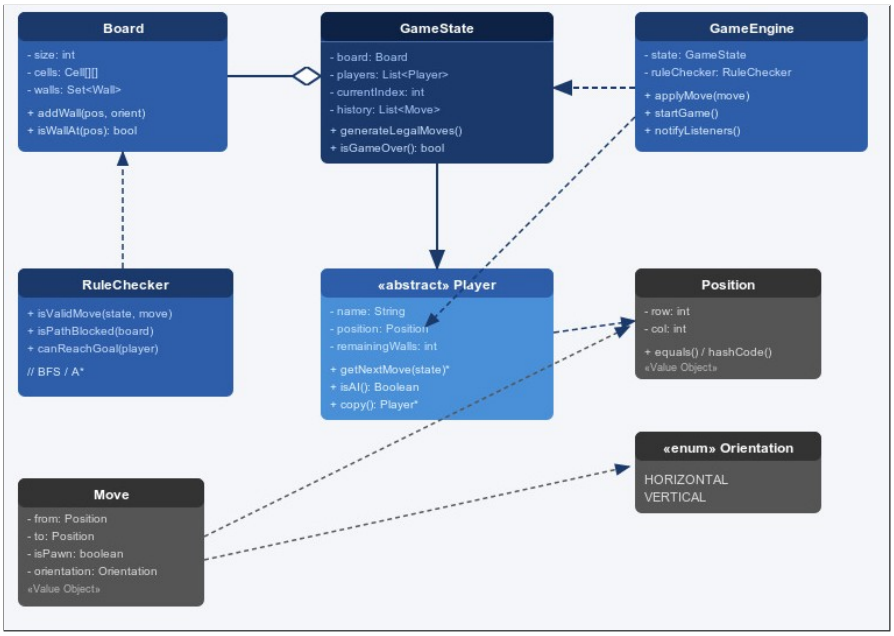


FIGURE 2 – Une architecture en couches

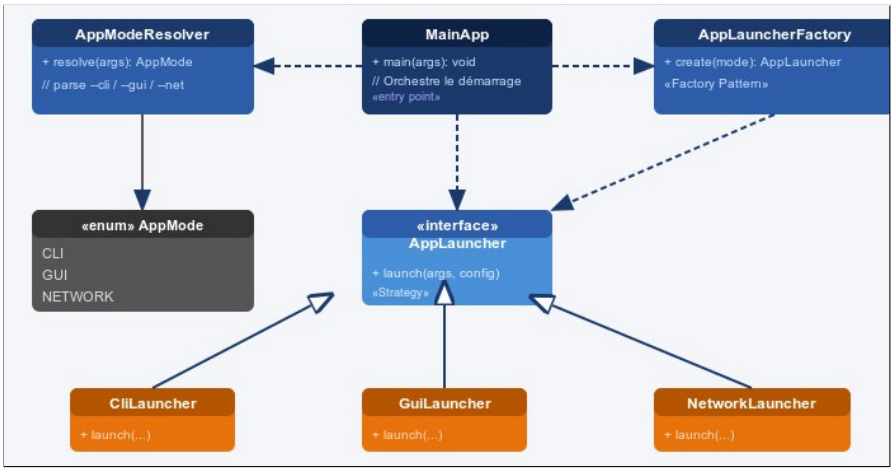


FIGURE 3 – Une architecture en couches

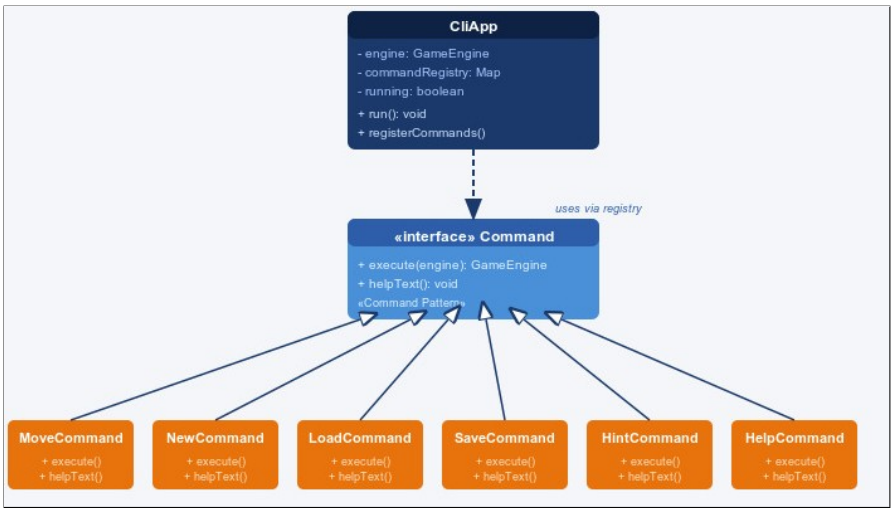


FIGURE 4 – Une architecture en couches

— **Builder pour MinimaxPlayer** : configuration déclarative et lisible de l'IA

1.10 Axes d'amélioration

Références

[1] Cameron Browne. Bitboard methods for games. *ICGA Journal*, 37(2) :67–84, June 2014.

[2] Robert C. Martin. *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. Prentice Hall, Boston, MA, USA, 2017.

A Section en annexe

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.